

Proyecto #1 — Competencia de Modelación (MLP · Ames House Prices)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

1. Análisis exploratorio de datos (EDA)

1.1 Dimensiones y tipos de variables

`train.csv` tiene 1168 filas × 81 columnas. La variable objetivo es `SalePrice` (continua, USD). De las 79 features restantes (se descarta `Id`, identificador sin señal): 36 numéricas (áreas, conteos, años) y 43 categóricas de texto. Dentro de las categóricas, 20 son en realidad **ordinales** con un orden natural (calidad: `Po<Fa<TA<Gd<Ex`; exposición de sótano; terminación de garaje; pendiente del terreno; etc.) y 23 son **nominales puras** sin orden (`Neighborhood`, `SaleType`, `Exterior1st`, ...).

1.2 Estadísticas descriptivas

`SalePrice`: media \$181,442, mediana \$165,000, desviación estándar \$77,264, rango [\$34,900, \$745,000]. Las features numéricas con mayor correlación con el precio son `OverallQual` (calidad general de construcción), `GrLivArea` (área habitable), `GarageCars`/`GarageArea`, `TotalBsmtSF` y `1stFlrSF` — todas relacionadas con tamaño y calidad de la construcción, consistente con la intuición de mercado inmobiliario. `OverallCond` (condición) correlaciona casi nulo, sugiriendo que la calidad de construcción pesa más que el estado de mantenimiento.

1.3 Valores nulos, outliers e inconsistencias

19 columnas tienen nulos. En este dataset, NA casi siempre significa “**la casa no tiene esa característica**” (sin piscina, sin garaje, sin sótano) según el diccionario de datos original de Ames — no es un dato faltante real. Tratamiento:

- `PoolQC` (1162 nulos), `MiscFeature` (1122), `Alley` (1094), `Fence` (935), `FireplaceQu` (547), y las columnas `Garage*/Bsmt*/MasVnrType`: nulo → categoría explícita “None”. No se eliminan columnas pese al alto % de nulos, porque el nulo mismo es informativo (ausencia de la característica correlaciona con precio).
- `LotFrontage` (217 nulos, numérica): sin ausencia estructural clara → imputación por mediana.
- `Electrical` (1 nulo): imputación por moda.

Outliers: se identificaron 2 casas con `GrLivArea` > 4000 y `SalePrice` < \$300,000 (`Id` 524 y 1299) — ventas atípicas documentadas del dataset original de Ames que rompen la relación esperada área↔precio. Se eliminan del set de entrenamiento (quedan 1166 filas).

1.4 Visualizaciones

Ver `docs/figs/`: `saleprice_dist.png` (distribución de `SalePrice`, sesgo positivo marcado, y su versión `log1p` mucho más simétrica), `outliers_scatter.png` (`GrLivArea`/`OverallQual` vs. `SalePrice`, outliers visi-

bles), correlaciones.png (top features numéricas correlacionadas con el precio).

1.5 Decisiones de preprocesamiento (resumen)

Decisión	Justificación
Eliminar Id	identificador, sin señal
Eliminar 2 outliers GrLivArea>4000 & SalePrice<\$300k	ventas atípicas conocidas del dataset
Target $\rightarrow \log_{1p}(\text{SalePrice})$	corrige sesgo positivo; el error penaliza proporcionalmente en toda la escala
Nulos en categóricas de ausencia de feature $\rightarrow$ "None"	el nulo es informativo
LotFrontage $\rightarrow$ mediana; Electrical $\rightarrow$ moda	únicos nulos sin ausencia estructural clara
20 ordinales de calidad $\rightarrow$ OrdinalEncoder con orden real por columna	preserva el orden, evita explotar dimensionalidad
23 nominales puras $\rightarrow$ one-hot	sin orden natural
36 numéricas $\rightarrow$ StandardScaler	el MLP converge mejor con features en escala similar

1.6 Feature engineering

Además del preprocesamiento de columnas crudas, se agregan features derivadas (engineer_features en src/preprocessing.py), calculadas **después** de excluir outliers para no derivar de filas que luego se descartan:

Feature derivada	Fórmula	Motivación
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	área total, más correlacionada con el precio que cualquier componente por separado
HouseAge	YrSold - YearBuilt	antigüedad, más interpretable que el año crudo
RemodAge	YrSold - YearRemodAdd	años desde la última remodelación
TotalBath	FullBath + 0.5·HalfBath + BsmtFullBath + 0.5·BsmtHalfBath	conteo total de baños ponderado
Qual_x_TotalSF	OverallQual · TotalSF	interacción calidad×área, las dos señales más fuertes
OverallGrade	OverallQual · OverallCond	composite calidad×condición
TotalPorchSF	suma de las 5 áreas de porche/deck	superficie exterior total
Has* (garaje, sótano, 2º piso, piscina, chimenea), IsRemodeled	flags binarios	codifican explícitamente la ausencia de característica
*_log	log1p de 9 áreas sesgadas	linealizan magnitudes con sesgo positivo fuerte

Tras codificación, el vector de entrada al MLP tiene ~245 dimensiones.

2. Metodología de desarrollo

2.1 Arquitectura del MLP

El modelo es un **perceptrón multicapa** (MLP en `src/model.py`): una capa oculta de **96 neuronas** con activación ReLU y dropout 0.1, más una **conexión residual lineal (skip connection)** que suma al resultado una transformación lineal directa de la entrada. La salida es una única neurona (regresión sobre `log1p(SalePrice)`).

$$\hat{y} = W_{out} \text{ReLU}(W_{hidden} x) + W_{skip} x$$

La skip connection es la decisión de arquitectura de mayor impacto: la relación entre las features y el precio (en log-espacio) es fuertemente lineal, y darle a la red un camino lineal explícito le permite capturar esa componente directamente y dedicar la capa oculta solo a la corrección no-lineal, en vez de tener que reconstruir la parte lineal con ReLU sobre ~1000 muestras.

2.2 Entrenamiento

- **Pérdida:** `MSELoss` sobre el target en escala `log1p(SalePrice)`. Trabajar en log evita que las pocas casas caras dominen el gradiente y hace que el error sea proporcional en toda la escala de precios.
- **Estandarización del target:** además del log, el target se **estandariza** (z-score con la media y desviación del fold de entrenamiento) antes de entrenar; la predicción se invierte en inferencia. Esto mejora la optimización de Adam.
- **Optimizador:** Adam, learning rate inicial 1e-3, con `ReduceLR0nPlateau` (baja el LR a la mitad cuando el RMSE de validación se estanca) para afinar la convergencia.

- **Regularización:** dropout 0.1 y **early stopping** por RMSE de validación (paciencia 30 épocas, restaura los pesos del mejor punto), sobre un máximo de 400 épocas. Batch size 32.

2.3 Validación y modelo final: ensemble de MLPs

La configuración se valida con **5-fold cross-validation**: en cada fold se reajusta el pipeline de preprocesamiento solo con los datos de ese fold de train (evitando fuga) y se entrena un MLP desde cero. Para reducir la varianza de la estimación y del modelo, la CV se repite con **5 semillas** distintas (42, 1, 7, 11, 23).

El **modelo entregado es el ensemble de los 25 MLPs** resultantes (5 folds × 5 semillas): `predict.py` promedia sus predicciones en log-espacio antes de invertir a USD. Cada MLP vio ~80% de los datos y fue validado contra el 20% restante, así que el ensemble no cuesta entrenamiento extra respecto a la validación cruzada.

`predict.py` también aplica un **clip de seguridad** $[0.5 \times \min(\text{SalePrice}), 1.5 \times \max(\text{SalePrice})]$ observado en train, para acotar el daño de una extrapolación catastrófica al invertir $\log \rightarrow \text{USD}$ sobre una casa fuera de rango (sección 4).

3. Resultados de iteraciones

Todas las métricas son **RMSE en USD por 5-fold cross-validation multi-seed** (media sobre las semillas), la estimación más fiel del desempeño en datos nuevos.

Iteración	Cambio respecto a la anterior	Val RMSE (USD)
I1	MLP base (1 capa, ReLU) + feature engineering	~\$25,800
I2	+ skip connection + estandarización del target + <code>ReduceLROnPlateau</code>	~\$19,100
I3	Ajuste del ancho de capa (48 → 64 → 80 → 96)	\$18,989
I4	Ensemble 5 semillas × 5 folds (25 MLPs)	\$18,795

Detalle del ajuste de ancho (I3), con skip + estandarización + scheduler:

Ancho capa oculta	Val RMSE (USD)
48	\$19,905
64	\$19,055
80	\$19,144
96	\$18,989

Evidencia de overfitting con capacidad excesiva: al aumentar la capacidad sin regularización adecuada (2–3 capas, o anchuras de 128–512 neuronas), el gap train/validación crece y el RMSE de validación empeora de forma marcada — con ~1000 muestras y ~245 features, la razón parámetros/muestras es alta y la red memoriza. El dropout, la skip connection y el early stopping son los que permiten que un ancho de 96 generalice mejor que uno menor sin sobreajustar.

4. Discusión de resultados

Qué cambios tuvieron mayor impacto. El salto grande (I1→I2, de ~\$25.8k a ~\$19.1k) viene de tres cambios que atacan el mismo problema: que un MLP con ReLU tiene dificultad para representar con precisión la relación (esencialmente lineal en log-espacio) entre features y precio a partir de pocas muestras. La **skip connection** le da esa componente lineal directamente; la **estandarización del target** y el **scheduler** mejoran la calidad de la optimización. El ajuste de ancho (I3) y el **ensemble** (I4) aportan mejoras menores pero consistentes por reducción de varianza.

Análisis de errores del modelo final. Los errores más grandes se concentran en casas de valor alto o atípico; el modelo generaliza peor en los extremos de la distribución de precio/tamaño que en el rango típico (\$130k–\$215k), donde está la mayoría de los datos de entrenamiento. Esto es esperable: hay pocas observaciones en los extremos para aprender su comportamiento.

Fragilidad ante extrapolación (y su mitigación). Al invertir log→USD con `expm1`, un error moderado en log-espacio sobre una casa con área muy fuera del rango de entrenamiento se amplifica exponencialmente en dólares. El clip de seguridad `[0.5×min, 1.5×max]` acota ese daño. Sigue siendo una limitación: el clip acota, no elimina; si el held-out contiene casas muy fuera del rango de `train.csv`, el error en esos puntos será alto.

Trade-off complejidad/generalización. El hallazgo central es que, con este dataset, la mejora no vino de *más* capacidad sino de una arquitectura y un entrenamiento **mejor condicionados**: la skip connection alinea el modelo con la estructura lineal de los datos, y la regularización (dropout, early stopping, ensemble) mantiene baja la complejidad efectiva. Subir capacidad sin esto solo aumentó el overfitting.

5. Conclusiones

- **Desempeño final:** el ensemble de 25 MLPs alcanza **RMSE ≈ \$18,795 USD** en validación cruzada multi-seed, y **RMSE = \$20,881 USD** sobre el dataset de prueba de la competencia. La cercanía entre ambos (CV vs. held-out real) confirma que la estimación por CV es fiable y que el modelo generaliza sin sobreajuste apreciable.
- **Aprendizajes técnicos:** (1) entrenar sobre el target en escala log, y además estandarizarlo, es determinante para la optimización del MLP en datos de precios con sesgo positivo; (2) una **skip connection** puede ser más valiosa que agregar capacidad cuando la relación subyacente es fuertemente lineal; (3) con pocas muestras relativas a la dimensión, la regularización correcta (dropout + early stopping + ensemble) generaliza mejor que una red más grande sin regularizar; (4) validar con k-fold CV multi-seed da una estimación de generalización fiable y barata; (5) el ensemble de los modelos de CV es prácticamente gratis y reduce la varianza del modelo final; (6) invertir una transformación no lineal (`expm1`) amplifica errores de forma no uniforme, por lo que un clip de seguridad simple es una red de contención útil.
- **Mejoras futuras:** más feature engineering (interacciones adicionales); un modelo de incertidumbre (cuantiles/intervalos) en vez del clip fijo; y arquitecturas de MLP con normalización o conexiones residuales más profundas si se dispusiera de más datos.

6. Enlace al repositorio de GitHub

<https://github.com/AscencioSIUU/deepLearning/tree/main/projects/proy-1-competencia>

El repositorio contiene el código completo: preprocesamiento (`src/preprocessing.py`), la arquitectura del MLP y el training loop (`src/model.py`), el entrenamiento del ensemble por validación cruzada (`src/ensemble.py`, `train.py`) y la predicción/evaluación (`predict.py`), con instrucciones de reproducción en el `README.md` del proyecto.